Software de Aerolíneas Low-Cost

Joaquin Ferramola

Escuela Técnica N°32 DE 14 “Gral. José de San Martin”, CABA

Proyecto Interdisciplinario

Nahuel Acosta y

Gabriel Abello Tuinyla

CABA, Argentina

6 de Agosto de 2026

La presente carpeta de campo tiene como objetivo registrar de manera sistemática y detallada las actividades realizadas por Joaquin Ferramola en el marco de las unidades curriculares: Programación Orientada a Objetos, Redes, Análisis de Sistemas, Base de Datos y Proyecto Informático II. El propósito principal es consolidar un control organizado de las tareas individuales y grupales, documentando con rigurosidad el proceso de desarrollo, los hitos alcanzados y las resoluciones técnicas ante las dificultades encontradas.

Cada entrada de la bitácora presentará una estructura normalizada que incluirá: fecha, localización, franja horaria de ejecución y una descripción objetiva de la actividad. Asimismo, se incorporarán observaciones y comentarios técnicos que permitan reflexionar sobre la evolución del proyecto y la toma de decisiones arquitectónicas.

Este documento se constituye como la evidencia formal del compromiso, participación y desempeño técnico de Joaquin Ferramola, sirviendo como instrumento de auditoría para evaluar de forma clara el progreso y la maduración del software durante el período establecido.

Inicio del Proyecto: Arquitectura General de la Solución

El presente proyecto integrador se compone de cuatro infraestructura tecnológicas interconectadas que dan soporte al modelo operativo de la aerolínea:

1. **Arquitectura de Base de Datos (MySQL)**

Diseño de un modelo relacional centralizado compuesto por 13 tablas/entidades principales: Aeropuerto, Avion, Departamento, Servicio, Usuario, Pasajero, Empleado, Asiento, Vuelo, Boleto, Boleto_Servicio, Pago y CheckIn. La lógica de persistencia cuenta con 130 procedimientos almacenados (Stored Procedures) para la manipulación de datos y 130 disparadores (Triggers). Los disparadores se parametrizan predominantemente bajo la condición BEFORE DELETE para garantizar la integridad referencial y actuar como restricciones de seguridad activa, impidiendo fallos de consistencia en cascada al intentar eliminar registros dependientes (por ejemplo, la supresión de un pasajero con boletos activos asociados).

2. **Aplicación de Gestión Interna (Java - Desktop)**

Desarrollo de una aplicación de escritorio orientada a objetos con interfaces gráficas específicas para el personal operativo y los administradores de la empresa. Este entorno interactúa mediante conectividad JDBC con la base de datos centralizada, restringiendo por completo el acceso a usuarios externos o clientes. Implementa módulos CRUD (Create, Read, Update, Delete) completos para las entidades Vuelos, Boletos, Asientos, CheckIns, Pagos, Pasajeros, Empleados y Cuentas. El backend incorpora capas de validación y manejo de excepciones locales que bloquean la inserción o edición de campos con valores incompatibles con las restricciones y tipos de datos nativos de la base de datos.

3. **Plataforma Web Comercial y Operativa (HTML, CSS y JavaScript)**

Desarrollo de un portal web frontend desacoplado que segmenta la experiencia de usuario en dos perfiles diferenciados: Clientes y Empleados (prescindiendo de un entorno web para el administrador). La interfaz de clientes habilita exclusivamente un flujo de navegación asincrónico para la consulta de itinerarios y la adquisición autónoma de boletos. Por su parte, la vista de empleados integra herramientas CRUD completas para la gestión en tiempo real de Vuelos y Boletos desde cualquier navegador web.

4. **Infraestructura y Topología de Redes (Cisco Packet Tracer)**

Diseño y simulación de una infraestructura de red corporativa que interconecta un total de 350 terminales de trabajo (PCs) mediante 20 routers distribuidos jerárquicamente por sectores operativos. Cada nodo sectorial cuenta con un router Cisco 4331 enlazado a un switch multicapa Cisco 3650, el cual se conecta a un switch de acceso Cisco 2960. Las terminales (con una densidad variable de entre 6 y 20 computadoras por sector) se vinculan a este últjuanimo mediante una topología física en estrella para optimizar el rendimiento del canal de comunicación. Los sectores con requerimientos de alta disponibilidad y criticidad operativa incorporan un segundo router Cisco 2960 conectado a subredes redundantes de 6 computadoras con su propio switch de acceso, conformando una topología en malla que asegura la tolerancia a fallos en el transporte de datos.

Fecha 30/07/2026

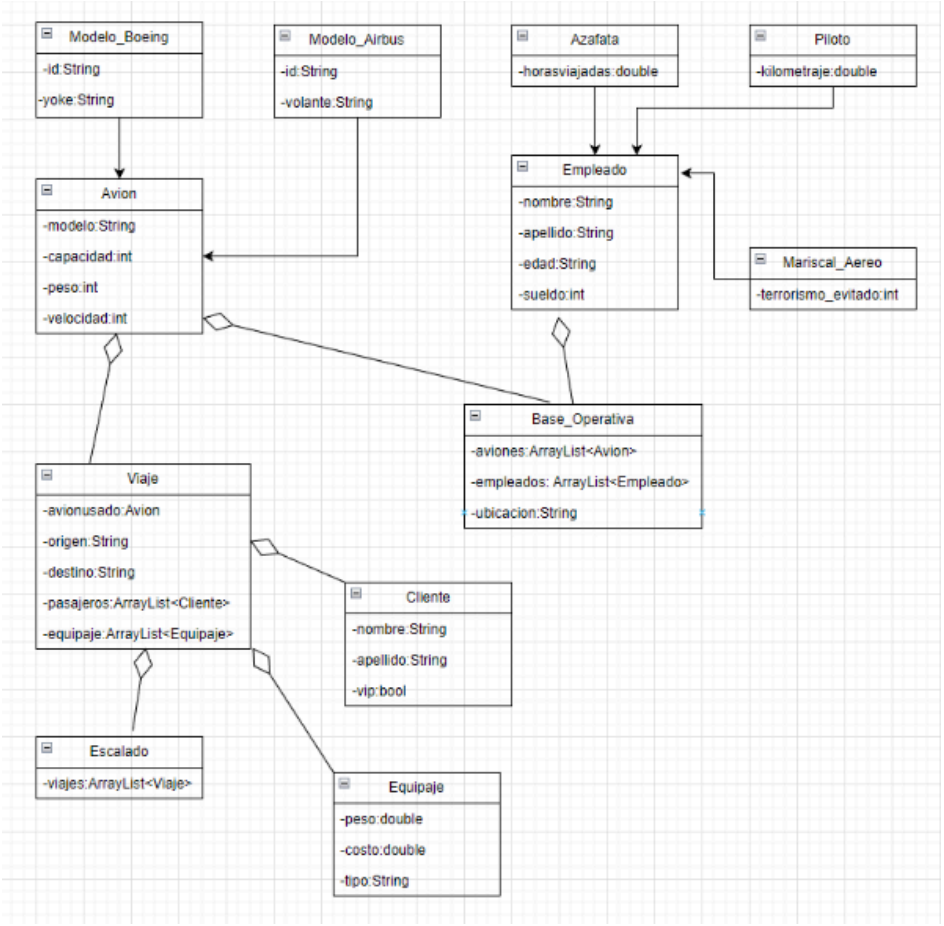
Diseñamos el primer modelo preliminar del diagrama de clases UML para la aplicación de Java. Esta versión inicial se elaboró de forma conceptual antes de contar con las

especificaciones técnicas definitivas del alcance del proyecto, por lo que sirvió como base de discusión y fue posteriormente descartada para optimizar la arquitectura del sistema:

En la estructura original, se había planteado una clase padre "Avión" con múltiples clases hijas especializadas. Tras analizar los requerimientos de la base de datos, se resolvió suprimir la herencia y unificar la entidad en una clase individual única llamada "Avion", parametrizando la flota inicial en exactamente cuatro registros de aeronaves dentro del sistema.

De igual manera, descartamos la estructura de herencia para la entidad "Empleado" y sus clases hijas derivadas. En su lugar, implementamos un modelo de asociación mediante la creación de la clase "Departamento". De este modo, los tipos de roles (pilotos, seguridad, administración, etc.) quedan normalizados como atributos de nombre dentro de cada departamento, simplificando el backend.

Finalmente, realizamos una refactorización de términos en el diagrama de clases: la entidad "Viaje" fue reemplazada de forma definitiva por la clase "Vuelo" y se procedió a la adición de nuevas clases complementarias necesarias para cubrir la lógica de negocio del MVP.



Fecha: 31/07/2026**:**

Durante este día y los siguientes, estuve trabajando junto con mi compañero Juan Molinas para armar el diagrama de gantt de todo el proyecto. Nos llevó varios días terminarlo porque tuvimos que sentarnos a hacer una lista con todas las cosas que necesitábamos para crear el sistema,

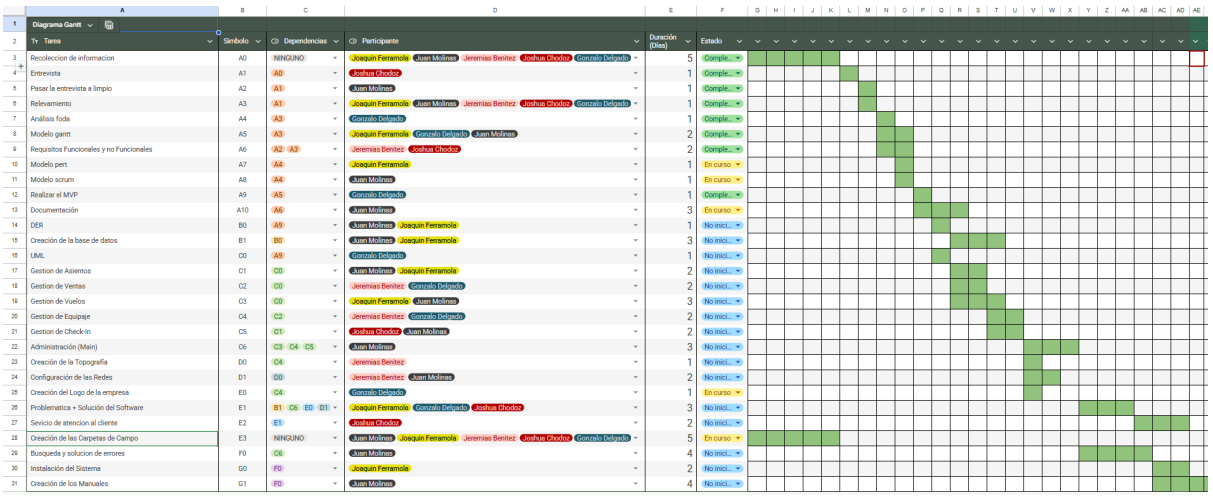


Diagrama Gantt					
Tarea	Símbolo	Dependencias	Participante	Duración (Días)	Estado
Recoleccion de informacion	A0	NINGUNO	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodos, Gonzalo Delgado	5	Comple...
Entrevista	A1	A0	Joshua Chodos	1	Comple...
Pasar la entrevista a limpio	A2	A1	Juan Molinas	1	Comple...
Relevamiento	A3	A1	Joaquin Ferramola, Juan Molinas, Jeremias Benitez, Joshua Chodos, Gonzalo Delgado	1	Comple...
Análisis foda	A4	A3	Gonzalo Delgado	1	Comple...
Modelo gantt	A5	A3	Joaquin Ferramola, Gonzalo Delgado, Juan Molinas	2	Comple...
Requisitos Funcionales y no Funcionales	A6	A2, A3	Jeremias Benitez, Joshua Chodos	2	Comple...
Modelo pert	A7	A4	Joaquin Ferramola	1	En curso
Modelo scrum	A8	A4	Juan Molinas	1	En curso
Realizar el MVP	A9	A5	Gonzalo Delgado	1	Comple...
Documentación	A10	A6	Juan Molinas	3	En curso
DER	B0	A9	Juan Molinas, Joaquin Ferramola	1	No inici...
Creación de la base de datos	B1	B0	Juan Molinas, Joaquin Ferramola	3	No inici...
UML	C0	A9	Gonzalo Delgado	1	No inici...
Gestion de Asientos	C1	C0	Juan Molinas, Joaquin Ferramola	2	No inici...
Gestion de Ventas	C2	C0	Jeremias Benitez, Gonzalo Delgado	2	No inici...
Gestion de Vuelos	C3	C0	Joaquin Ferramola, Juan Molinas	3	No inici...
Gestion de Equipaje	C4	C2	Jeremias Benitez, Gonzalo Delgado	2	No inici...
Gestion de Check-In	C5	C1	Joshua Chodos, Juan Molinas	2	No inici...
Administración (Main)	C6	C3, C4, C5	Juan Molinas	3	No inici...
Creación de la Topografía	D0	C4	Jeremias Benitez	1	No inici...
Configuración de las Redes	D1	D0	Jeremias Benitez, Juan Molinas	2	No inici...
Creación del Logo de la empresa	E0	C4	Gonzalo Delgado	1	En curso
Problematica + Solución del Software	E1	B1, C6, E0, D1	Joaquin Ferramola, Gonzalo Delgado, Joshua Chodos	3	No inici...
Sevicio de atencion al cliente	E2	E1	Joshua Chodos	2	No inici...
Creación de las Carpetas de Campo	E3	NINGUNO	Juan Molinas, Joaquin Ferramola, Jeremias Benitez, Joshua Chodos, Gonzalo Delgado	5	En curso
Busqueda y solucion de errores	F0	C6	Juan Molinas	4	No inici...
Instalación del Sistema	G0	F0	Joaquin Ferramola	2	No inici...
Creación de los Manuales	G1	F0	Juan Molinas	4	No inici...

Fecha: 4/08/2026

arranqué a armar el modelo PERT , usando como base toda la información y las tareas del diagrama de Gantt que ya teníamos armados. Básicamente, me puse a mirar la lista de cosas que teníamos que hacer para el proyecto y empecé a conectarlas (como si fuera un mapa conceptual) para ver cómo se enganchaba una tarea con la otra.

Fecha 05/08/2026

Se crea la carpeta de campo grupal en un documento de Google para centralizar el relevamiento de requerimientos del MVP. Diseñamos un cuestionario dividido en 5 campos estratégicos (Piloto, Control de Vuelos, Control Operativo, Encargado de Ganancias y

Analista de Costos) para estructurar la lógica del software. Las definiciones del día quedaron distribuidas de la siguiente manera:

En el módulo de Piloto, definimos las preguntas para relevar la duración de viajes, trayectos habituales, personal técnico en cabina por ruta, frecuencia de escalas, el impacto de las tasas aeroportuarias y los costos directos en el aire.

En el módulo de Gestión de Vuelos, establecemos las preguntas sobre la estructura de precios base, asignación de asientos, cobro de equipaje, viabilidad de escalas, algoritmos de rellenado automático, penalizaciones por cancelación de último momento y la verificación legal de habilitaciones para pilotos y azafatas.

En el módulo de Encargado de Ganancias, fijamos los cuestionamientos comerciales sobre el funcionamiento del algoritmo de precios dinámicos, el monitoreo técnico de la competencia y las variables matemáticas para la predicción de ventas futuras.

En el módulo de Control Operativo, estructuramos las preguntas para el diseño de la interfaz de la línea de vuelos activa (retrasos y cancelaciones), la lógica de la calculadora de impacto de demoras (efecto dominó) y el proceso logístico de asignación de aviones a puertas de embarque.

En el módulo de Analista de Costos, cerramos el documento con las preguntas enfocadas en el cálculo de desvíos financieros por factores externos (clima y combustible extra) y la medición de la ganancia real por asiento y kilómetro operativo.

En paralelo, realizamos la creación del repositorio remoto en GitHub para la gestión y el control de versiones distribuido del proyecto. Vinculamos a todos los integrantes del equipo de desarrollo al repositorio central, estableciendo el entorno colaborativo inicial y definiendo las pautas de confirmación de cambios (commits) para la futura integración del código del backend en Java, la interfaz web y los scripts de la base de datos MySQL.

Fecha 06/08/2026

Procesamos las respuestas del cuestionario de relevamiento y realizamos la bajada técnica de los requerimientos funcionales para el MVP, traduciendo la información del modelo operativo low-cost en las reglas de negocio que estructurarán el backend.

Para el módulo de Piloto, establecemos las limitaciones lógicas del sistema de asignación de rutas, definiendo un bloqueo automático para tramos mayores a 5 horas para forzar patrones en espejo. También configuramos los parámetros lógicos para las alertas visuales en rampa que controlarán el turnaround crítico de 30 minutos y los tiempos de descanso de la tripulación técnica.

En Gestión de Vuelos, fijamos la lógica de backend para la estrategia de unbundling (cobro desglosado de equipaje y servicios adicionales) y el diseño del algoritmo de dispersión intencional de asientos en el check-in gratuito. Asimismo, parametrizamos las reglas ultra-estrictas del Crew Management System para el bloqueo automatizado de tripulaciones por límites de horas legales y la reasignación inmediata de no-shows para personal en traslado (deadheading).

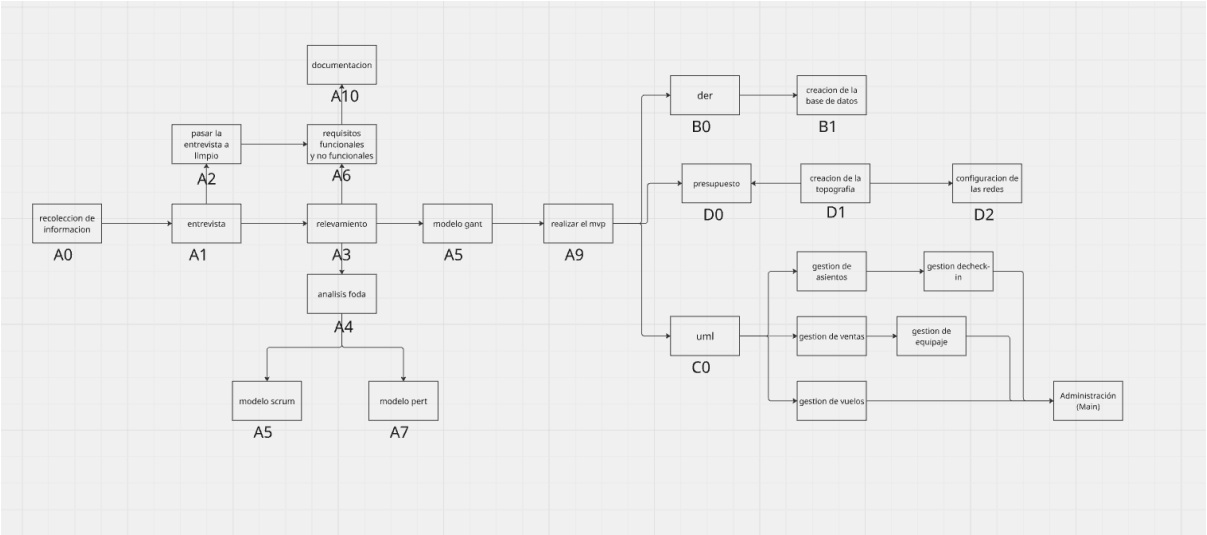
Para el Encargado de Ganancias, definimos la estructura matemática del motor de precios dinámicos según las curvas de reserva históricas para forzar un factor de ocupación del 90%. Adicionalmente, establecemos los requisitos técnicos de latencia, frecuencia y brecha de precios para el submódulo de inteligencia competitiva basado en web scraping.

En Control Operativo, determinamos el diseño de la interfaz gráfica del OCC basada en diagramas de Gantt de alta densidad alimentados por telemetría ACARS para actualizar retrasos en tiempo real. También estructuramos las variables de las calculadoras de impacto financiero (multas de slots, combustible, compensaciones) y la simulación del efecto dominó para predecir colapsos de jornadas (Duty Period Time Out).

Finalmente, para el Analista de Costos, estructuramos las fórmulas para las auditorías automatizadas de desvíos económicos en el aire (Actual vs Plan), aislando el combustible cargado por discrecionalidad del comandante. Cerramos definiendo la lógica de reportes ejecutivos para el cálculo automatizado de las densidades económicas RASK (incluyendo Ancillary RASK) y CASK.

Fecha:07 /08/2026

terminamos de armar el modelo PERT. Para lograrlo, hicimos una pausa en la parte técnica y nos dedicamos a hacer una lista con absolutamente todas las tareas que nos faltaban hacer para organizar bien nuestros tiempos.



Dedicamos la jornada a listar y estructurar la totalidad de las actividades pendientes del proyecto para establecer las bases de la planificación temporal. Suspendimos momentáneamente el desarrollo técnico para realizar el desglose de tareas y categorizarlas en cinco fases críticas, definiendo el alcance global del desarrollo:

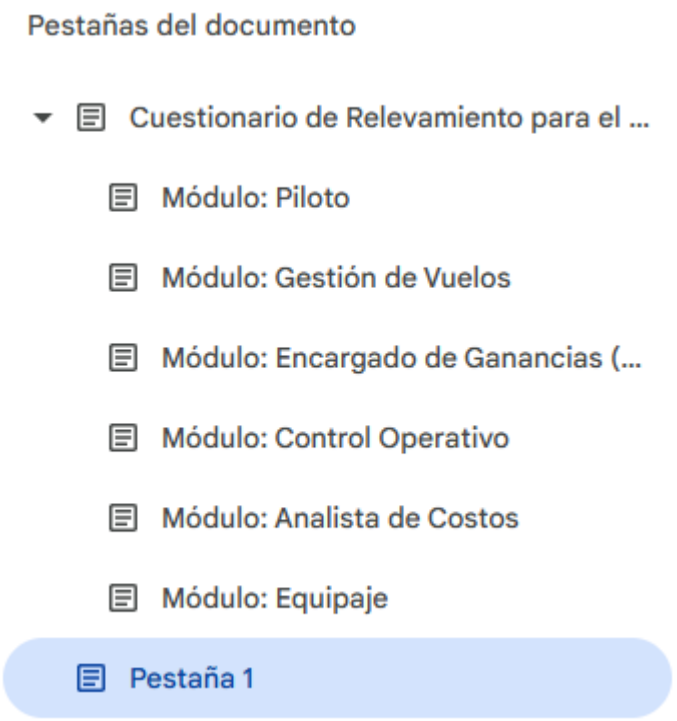
En la fase de Análisis, definimos las actividades de búsqueda de información (A1), el relevamiento final (A2), la creación del modelo Gantt, PERT, FODA y la metodología Scrum (A3), el listado de requisitos funcionales y no funcionales fijando un objetivo de 25 de cada uno (A4), la documentación técnica general (A5) y el cálculo matemático del Mínimo Producto Viable (A6).

En la fase de Diseño, establecemos las tareas de creación física de la base de datos (B1), el diseño detallado de la Interfaz de Usuario UI (B2), el diseño del logo corporativo de la empresa (B3), el armado del diagrama UML para programación orientada a objetos (B4) y la redacción del documento formal de presentación de la problemática junto a su solución (B5).

En la fase de Desarrollo, mapeamos los módulos que se programarán en la aplicación de Java y la web, los cuales incluyen la gestión de vuelos (C1), gestión de pasajeros (C2), gestión de ventas de boletos y precios (C3), gestión de equipaje (C4), creación de manuales técnicos (C5), servicio de atención al cliente (C6), el módulo de check-in (C7), el panel de administración (C8) y la actualización continua de la carpeta de campo grupal (C9).

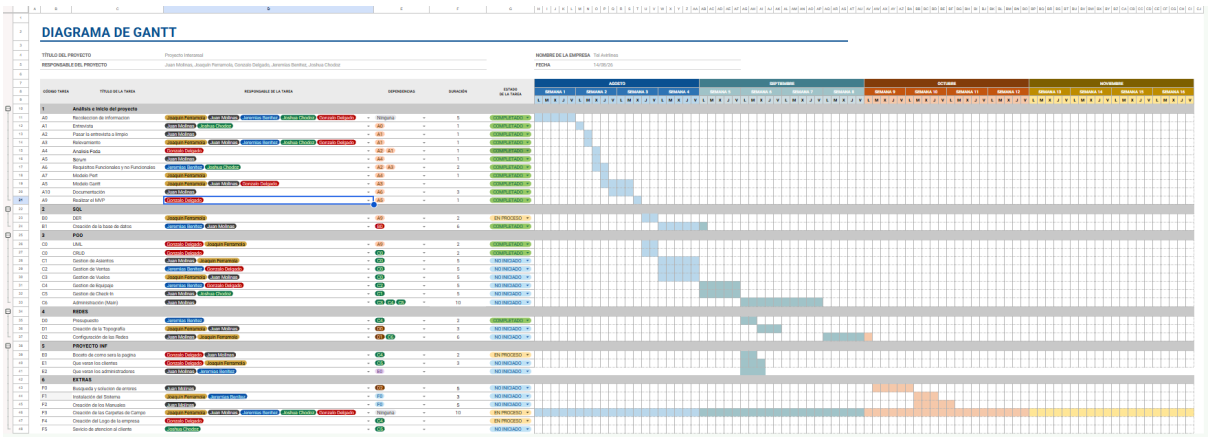
En las fases de Pruebas e Implementación, fijamos la actividad de búsqueda y depuración de errores del código (D1), el despliegue e instalación final del sistema completo, la entrega de manuales y la presentación final del proyecto (E1).

Finalmente, procesamos los requerimientos específicos del profesor para el diseño de la topología en Cisco Packet Tracer. Se registraron las densidades de red obligatorias por sector para el armado de la infraestructura de las aerolíneas: Check-in (10 PCs), Atención al Cliente (6 PCs), Operaciones (8 PCs), Equipaje (5 PCs) y Administración (5 PCs).
Ademas de todo eso se crearon pestañas en el documento para organizarlo de mejor manera y facilitar la busqueda del material de trabajo porque las respuestas del cuestionario ocupaban demasiado espacio.



Fecha:17 /08/2026

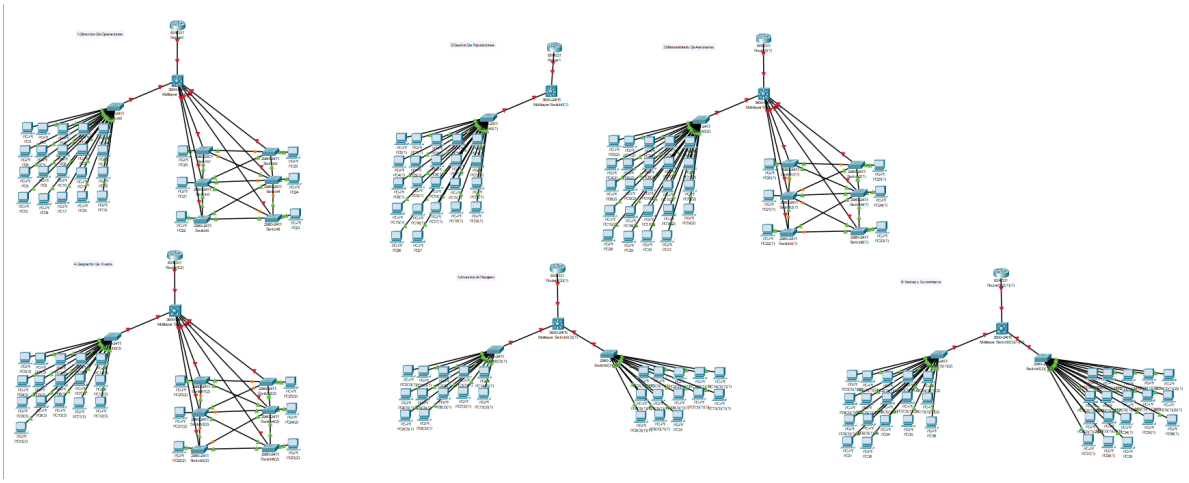
rmar la segunda versión de nuestro diagrama de Gantt. Aprovechamos esta etapa para terminar de ajustar bien nuestro calendario definitivo y dejar totalmente cerrada la asignación de las tareas para cada integrante del grupo.



(esta actividad duro varios días)

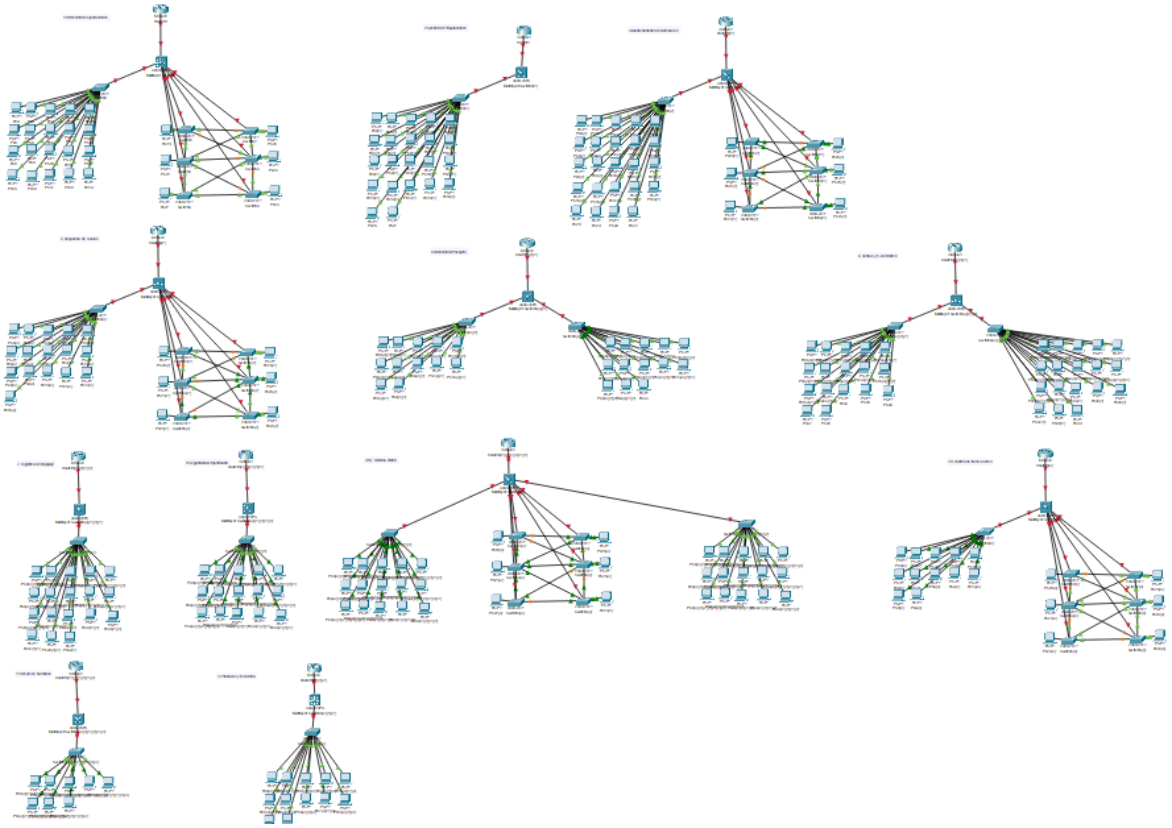
Fecha 01/09/2026

Ese día nos metimos de lleno en el Cisco Packet Tracer para empezar a armar toda la simulación de la red del proyecto. Estuvimos analizando y eligiendo qué equipos íbamos a usar para que todo funcione bien y tenga una estructura lógica y ordenada por sectores. Al final, decidimos armar la red utilizando un Router 4331 para cada sector principal, conectándolo a un Switch multicapa 3650 que actúa como el nodo central de esa red. De ese switch central colgamos varios switches 2960, que los usamos como los switches normales o "hijos" para conectar directamente a las computadoras de cada área.



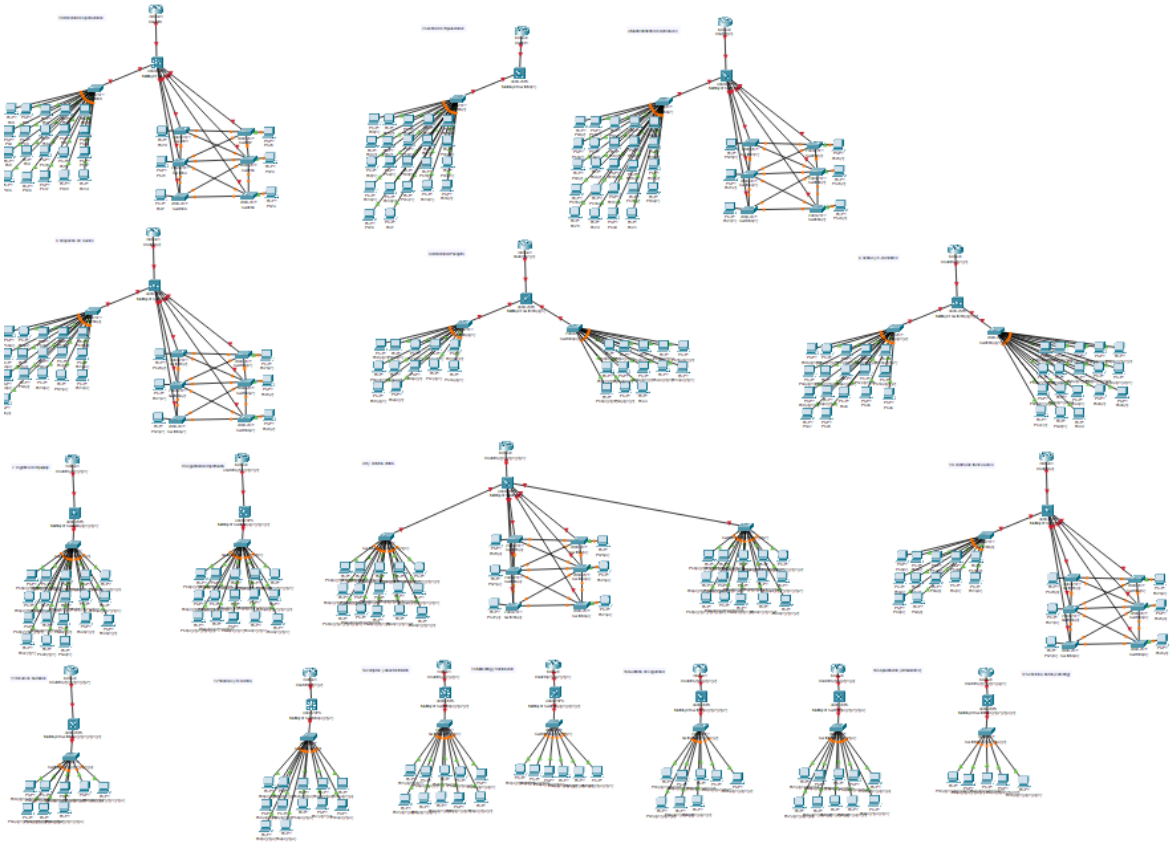
Fecha 02/09/26

le di continuidad al armado de la red dentro de cisco, enfocándonos de lleno en estructurar cada sector según la cantidad exacta de computadoras que necesitaba. Nos pusimos a configurar y distribuir las terminales de trabajo tomando en cuenta los requerimientos específicos de cada área de la aerolínea (como las terminales para check-in, atención al cliente, operaciones, equipaje y administración), asegurándonos de que cada sector tuviera conectados los dispositivos correctos a los switches y routers que ya habíamos definido previamente.



Fecha 10/09/26

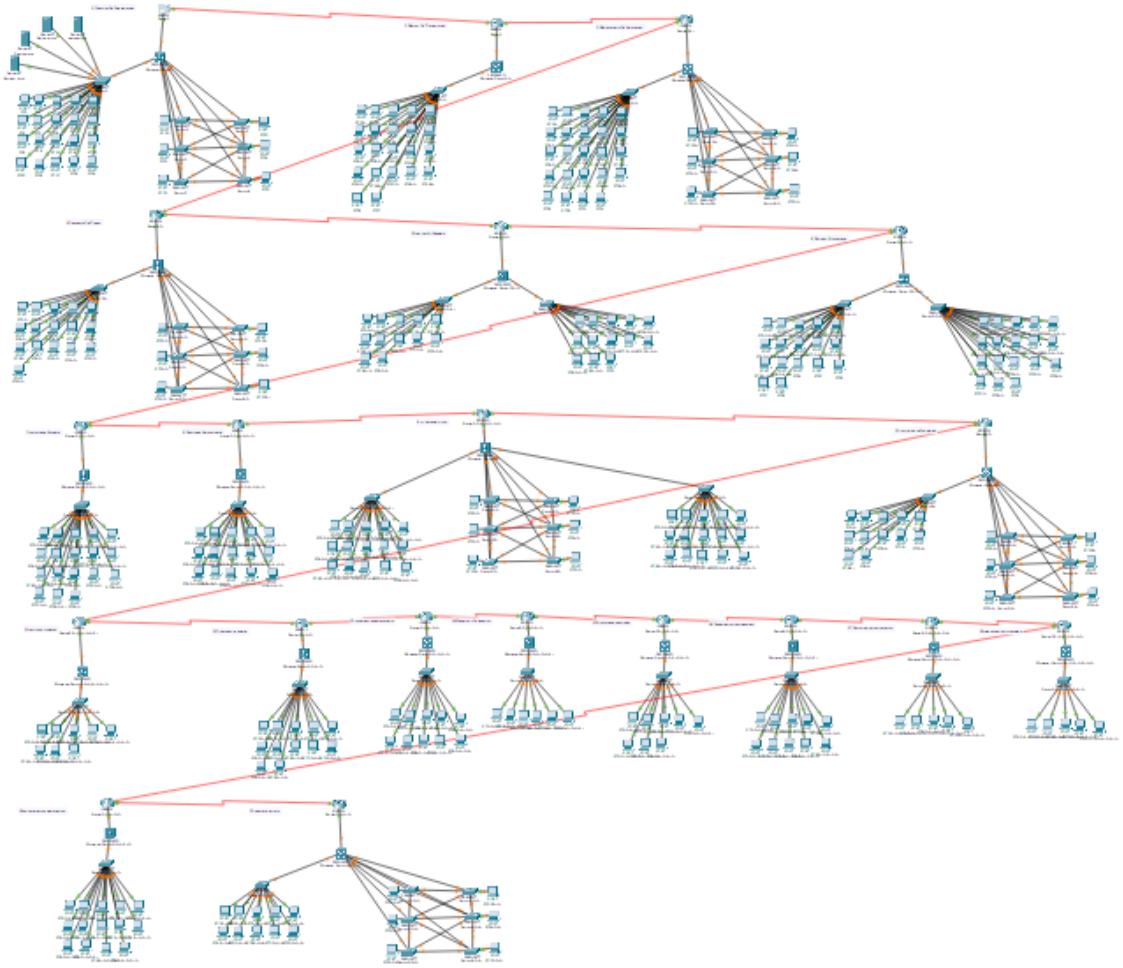
termine por completo de armar la topología de cada red dentro de Cisco Packet Tracer. Nos dedicamos a dejar listas todas las estructuras físicas de los sectores, pero decidimos que todavía no íbamos a interconectar las redes entre sí ni a configurar las direcciones IP o los parámetros de las máquinas. Por ahora



Fecha 16/09/26

termine la infraestructura de red y me puse a agregar y configurar todos los servidores que hacían falta para levantar los servicios principales, instalando detalladamente los servidores DNS, DHCP, FTP y HTTP para que la red empiece a tener los servicios lógicos operativos. Una vez que dejé eso montado, arranqué a conectar todos los routers entre sí con el objetivo de interconectar los diferentes sectores de la aerolínea. Pero en medio de las pruebas de conectividad se me presentó un inconveniente que me frenó un buen rato: cuando quise empezar a probar el funcionamiento del protocolo DHCP para que se encargue de asignar las direcciones IP de manera automática, me di cuenta de que la IP se le asignaba únicamente al primer router y de ahí no pasaba a los demás. Estuve revisando las conexiones y vi que esto significaba que la información, las solicitudes y las respuestas no se estaban propagando de forma

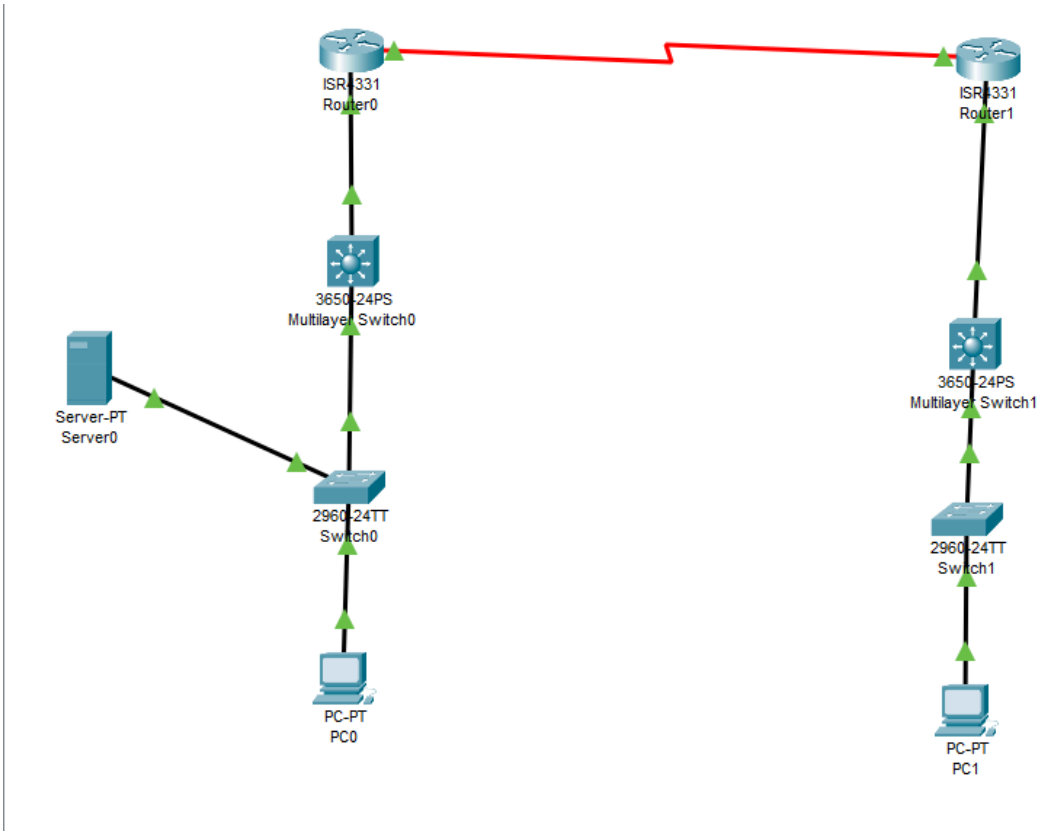
correcta a través de los equipos, por lo que tenía un problema claro en la comunicación y en el enrutamiento entre las redes que iba a tener que resolver



23/09/26

Ante el inconveniente que se me había presentado con el enrutamiento y la asignación de las direcciones IP, decidimos afrontar el problema junto con mi compañero Juan Molinas. Para poder solucionarlo sin arruinar lo que ya teníamos hecho, abrimos un archivo completamente

nuevo en el programa y nos pusimos a hacer pruebas desde cero. Fuimos probando distintas configuraciones y revisando paso a paso cómo se conectaban los routers hasta que le encontramos la vuelta para que la información y las asignaciones de red pasen de forma correcta entre los equipos.

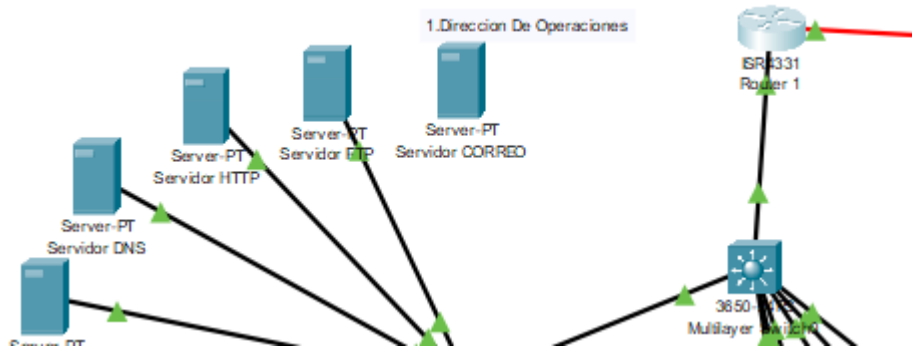


24/09/26

Una vez que pudimos solucionar el problema con la ayuda de mi compañero Juan Molinas, me puse a revisar que todo funcionara como correspondía para comprobar que la red ya no tuviera fallas. Después de eso, avancé con la configuración final y me encargué de asignarle de forma correcta la dirección IP a cada una de las máquinas de la simulación. Me aseguré de que cada router tenga específicamente configurada su propia IP para que todo quede bien ordenado y sectorizado dentro del sistema.

08/10/26

Luego de la explicación del profesor sobre cómo usar el servidor de correo, nos pusimos a implementarlo en nuestro trabajo de red para dejarlo funcionando por completo, quedándonos en total con los 5 servidores configurados: el servidor DNS, el servidor DHCP, el servidor FTP, el servidor HTTP y el servidor de correo (Mail). Una vez que dejamos listos todos los servidores de la topología, pasé a inventar una dirección de correo electrónico única y específica para cada una de las máquinas del sistema, los cuales más tarde mi compañero Joshua Chodos se encargaría de asignar y configurar de forma individual en cada computadora de la red.



05/10/26

Ese día me puse a pensar y diseñar los procedimientos almacenados para la base de datos del sistema, teniendo en cuenta la regla que nos habíamos marcado de tener exactamente 10 procedimientos por cada una de las tablas del modelo relacional. Me puse a analizar qué operaciones específicas iba a necesitar cada entidad para poder estructurarlos y dejarlos listos para su posterior programación en SQL.

Fecha 06/10/26

```
-- ----------------------------------------
-- PROCEDIMIENTOS DE Aeropuerto
-- ----------------------------------------
DELIMITER //
CREATE PROCEDURE Crear_Aeropuerto(IN p_nombre VARCHAR(120), IN p_pais VARCHAR(30), IN p_ciudad VARCHAR(80))
BEGIN
    INSERT INTO Aeropuerto(nombre, pais, city) VALUES(p_nombre, p_pais, p_ciudad);
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Aeropuerto(IN p_idaeropuerto INT, IN p_nombre VARCHAR(120), IN p_pais VARCHAR(30), IN p_ciudad VARCHAR(80))
BEGIN
    UPDATE Aeropuerto SET nombre = COALESCE(p_nombre, nombre), pais = COALESCE(p_pais, pais), ciudad = COALESCE(p_ciudad, ciudad) WHERE IDAeropuerto = p_idaeropuerto;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Aeropuertos()
BEGIN
    SELECT * FROM Aeropuerto;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Aeropuerto(IN p_idaeropuerto INT)
BEGIN
    SELECT * FROM Aeropuerto WHERE IDAeropuerto = p_idaeropuerto;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Aeropuerto(IN p_idaeropuerto INT)
BEGIN
    DELETE FROM Aeropuerto WHERE IDAeropuerto = p_idaeropuerto;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Aeropuerto_Por_Pais(IN p_pais VARCHAR(30))
BEGIN
    SELECT * FROM Aeropuerto WHERE pais = p_pais;
END//
```

Trabaje en los procedimientos de la tabla Aeropuerto

Crear_Aeropuerto: Sirve para dar de alta un nuevo aeropuerto en la base de datos recibiendo su nombre, país y ciudad como parámetros de entrada (IN).

Editar_Aeropuerto: Permite modificar los datos de un aeropuerto existente. Usa la función COALESCE para actualizar solo los campos que le envíes sin pisar los demás con valores nulos.

Mostrar_Aeropuertos: Devuelve una lista completa con todos los registros que estén guardados en la tabla Aeropuerto.

Buscar_Aeropuerto: Consulta y trae los datos de un aeropuerto en específico filtrándolo por su ID (IDAeropuerto).

Eliminar_Aeropuerto: Borra un registro de la tabla usando el ID del aeropuerto que se le pase.

Buscar_Aeropuerto_Por_Pais: Filtra y muestra todos los aeropuertos que pertenecen a un país determinado.

Fecha 12/10/26

termine los procedimientos de la tabla aeropuerto y empeze los de avion

Mostrar_Nombre_Aeropuertos: Devuelve una lista únicamente con los nombres de todos los aeropuertos guardados en la tabla.

Mostrar_Extras_Aeropuertos: Consulta y trae el ID y el nombre de los aeropuertos para mostrarlos de forma resumida.

Crear_Avion: Da de alta un nuevo avión en la base de datos recibiendo su ID, modelo y capacidad como parámetros de entrada.

Editar_Avion: Permite modificar los datos de un avión existente usando la función COALESCE para actualizar solo los campos enviados sin afectar con valores nulos.

Mostrar_Avion: Devuelve una lista completa con todos los registros de aviones guardados en la tabla.

```sql
DELIMITER //
CREATE PROCEDURE Mostrar_Nombre_Aeropuertos()
BEGIN
    SELECT nombre FROM Aeropuerto;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Extras_Aeropuertos()
BEGIN
    SELECT IDAeropuerto, nombre FROM Aeropuerto;
END//
DELIMITER ;

-- ----------------------------------------
-- PROCEDIMIENTOS DE Avion
-- ----------------------------------------
DELIMITER //
CREATE PROCEDURE Crear_Avion(IN p_idavion VARCHAR(30), IN p_modelo VARCHAR(50), IN p_capacidad INT)
BEGIN
    INSERT INTO Avion(IDAvion, modelo, capacidad) VALUES(p_idavion, p_modelo, p_capacidad);
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Avion(IN p_idavion VARCHAR(30), IN p_modelo VARCHAR(50), IN p_capacidad INT)
BEGIN
    UPDATE Avion SET modelo = COALESCE(p_modelo, modelo), capacidad = COALESCE(p_capacidad, capacidad) WHERE IDAvion = p_idavion;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Avion()
BEGIN
    SELECT * FROM Avion;
END//
DELIMITER ;
```

Fecha 13/10/26

Finalize la clase avion y empeze con la de asientos

Mostrar_ID_Aviones: Devuelve una lista únicamente con los identificadores de todos los aviones registrados en la tabla.

Buscar_Avion_Por_Capacidad: Filtra y muestra los aviones que coinciden exactamente con una capacidad de asientos específica.

Mostrar_Modelos_De_Aviones_Unicos: Devuelve una lista con los diferentes modelos de aviones sin repetir registros.

Buscar_Avion_Por_Rango_De_Capacidad: Permite buscar aviones filtrándolos dentro de un rango determinado de capacidad mínima y máxima.

Buscar_Asiento_Por_Id: Consulta y trae los datos de un asiento en específico utilizando su identificador único.

Buscar_Asientos_Por_Avion: Filtra y muestra todos los asientos que pertenecen a un avión determinado.

Mostrar_Disponibilidad_Asientos_Por_Avion: Devuelve el estado de disponibilidad de los asientos asociados a un avión específico.

Mostrar_Asientos: Muestra el listado completo con todos los registros de asientos guardados en la tabla.

Buscar_Asientos_Por_Disponibilidad: Filtra los asientos según su estado actual, indicando si se encuentran disponibles o no

. Editar_Asiento: Modifica el estado de disponibilidad de un asiento específico utilizando su identificador.

Ocupar_Asiento: Actualiza el estado de un asiento determinado para marcarlo como no disponible.

```sql
DELIMITER //
CREATE PROCEDURE Mostrar_ID_Aviones()
BEGIN
    SELECT IDAvion FROM Avion;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Avion_Por_Capacidad(IN p_capacidad INT)
BEGIN
    SELECT * FROM Avion WHERE capacidad = p_capacidad;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Modelos_De_Aviones_Unicos()
BEGIN
    SELECT DISTINCT modelo FROM Avion;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Avion_Por_Rango_De_Capacidad(IN p_capacidad_min INT, IN p_capacidad_max INT)
BEGIN
    SELECT * FROM Avion WHERE capacidad BETWEEN p_capacidad_min AND p_capacidad_max;
END//
DELIMITER ;

-- ----------------------------------------
-- PROCEDIMIENTOS DE Asiento
-- ----------------------------------------
DELIMITER //
CREATE PROCEDURE Buscar_Asiento_Por_Id(IN p_idasiento VARCHAR(50))
BEGIN
    SELECT * FROM Asiento WHERE IDAsiento = p_idasiento;
END//
DELIMITER ;
```

```sql
DELIMITER //
CREATE PROCEDURE Buscar_Asientos_Por_Avion(IN p_avion_id VARCHAR(30))
BEGIN
    SELECT * FROM Asiento WHERE avion_id = p_avion_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Disponibilidad_Asientos_Por_Avion(IN p_avion_id VARCHAR(30))
BEGIN
    SELECT disponible FROM Asiento WHERE avion_id = p_avion_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Asientos()
BEGIN
    SELECT * FROM Asiento;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Asientos_Por_Disponibilidad(IN p_disponible BOOLEAN)
BEGIN
    SELECT * FROM Asiento WHERE disponible = p_disponible;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Asiento(IN p_disponible BOOLEAN, IN p_idasiento VARCHAR(50))
BEGIN
    UPDATE Asiento SET disponible = p_disponible WHERE IDAsiento = p_idasiento;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Ocupar_Asiento(IN p_idasiento VARCHAR(50))
BEGIN
    UPDATE Asiento SET disponible = 0 WHERE IDAsiento = p_idasiento;
END//
DELIMITER ;
```

Fecha 19/10/26

Termine con la tabla asientos y empeze con la de departamento

```
DELIMITER //
CREATE PROCEDURE Mostrar_Departamento()
BEGIN
    SELECT * FROM Departamento;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Departamento(IN p_iddepartamento INT)
BEGIN
    SELECT * FROM Departamento WHERE IDDepartamento = p_iddepartamento;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Departamento(IN p_iddepartamento INT)
BEGIN
    DELETE FROM Departamento WHERE IDDepartamento = p_iddepartamento;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Departamento_Por_Nombre(IN p_nombre VARCHAR(100))
BEGIN
    SELECT * FROM Departamento WHERE nombre = p_nombre;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Departamento_Por_Tipo(IN p_tipo ENUM("Aéreo", "Terrestre"))
BEGIN
    SELECT * FROM Departamento WHERE tipo = p_tipo;
END//
```

Mostrar_Departamento: Devuelve una lista completa con todos los registros de departamentos guardados en la tabla.

Buscar_Departamento: Consulta y trae los datos de un departamento en específico filtrándolo por su ID.

Eliminar_Departamento: Borra un registro de la tabla utilizando el ID del departamento que se le pase.

Buscar_Departamento_Por_Nombre: Filtra y muestra los departamentos que coinciden con un nombre específico.

Buscar_Departamento_Por_Tipo: Filtra y muestra los departamentos según su tipo, ya sea Aéreo o Terrestre.

Fecha 20/10/26

Se termino la tabla departamentos y se le dio inicio a la tabla servicio

```
DELIMITER //
CREATE PROCEDURE Mostrar_Departamentos_Ordenados_Por_Nombre()
BEGIN
    SELECT * FROM Departamento ORDER BY nombre ASC;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Departamentos_Ordenados_Por_Nombre_Para_Atras()
BEGIN
    SELECT * FROM Departamento ORDER BY nombre DESC;
END//
DELIMITER ;

-- ========================================
-- PROCEDIMIENTOS DE Servicio
-- ========================================
DELIMITER //
CREATE PROCEDURE Crear_Servicio(IN p_nombre_servicio VARCHAR(20), IN p_precio DECIMAL(10, 2), IN p_descripcion VARCHAR(120))
BEGIN
    INSERT INTO Servicio(nombre_servicio, precio, descripcion) VALUES(p_nombre_servicio, p_precio, p_descripcion);
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Servicio(IN p_idservicio INT, IN p_nombre_servicio VARCHAR(20), IN p_precio DECIMAL(10, 2), IN p_descripcion VARCHAR(120))
BEGIN
    UPDATE Servicio SET nombre_servicio = COALESCE(p_nombre_servicio, nombre_servicio), precio = COALESCE(p_precio, precio), descripcion = COALESCE(p_descripcion, p_descripcion) WHERE IDServicio = p_idservicio;
END//
DELIMITER ;
```

Mostrar_Departamentos_Ordenados_Por_Nombre: Devuelve una lista con todos los departamentos ordenados alfabéticamente de manera ascendente por su nombre.

Mostrar_Departamentos_Ordenados_Por_Nombre_Para_Atras: Devuelve una lista con todos los departamentos ordenados alfabéticamente de manera descendente por su nombre.

Crear_Servicio: Da de alta un nuevo servicio en la base de datos recibiendo su nombre, precio y descripción como parámetros de entrada.

Editar_Servicio: Permite modificar los datos de un servicio existente utilizando la función COALESCE para actualizar únicamente los campos enviados sin afectar a los demás.

Eliminar_Servicio: Borra un registro de la tabla de servicios utilizando de forma directa el ID del servicio que se le pasa como parámetro.

Buscar_Servicio_Por_Nombre: Filtra y muestra los servicios que coinciden con un nombre específico.

Buscar_Servicio_Por_Precio: Consulta y trae los servicios que tienen un precio exacto al indicado.

Buscar_Servicio_Por_Descripcion: Filtra los servicios que coinciden con una descripción detallada específica.

Buscar_Servicio_Por_Precio_Mayor_A: Devuelve una lista con los servicios cuyo precio supera un monto determinado.

Fecha 21/10/26

Finalize la tabla servicios y empeze y termine con la tabla usuario

```
DELIMITER //
CREATE PROCEDURE Buscar_Servicio_Por_Rango_De_Precio(IN p_precio_min DECIMAL(10, 2), IN p_precio_max DECIMAL(10, 2))
BEGIN
    SELECT * FROM Servicio WHERE precio BETWEEN p_precio_min AND p_precio_max;
END//
DELIMITER ;

-- ==========================================
-- PROCEDIMIENTOS DE Usuario
-- ==========================================
DELIMITER //
CREATE PROCEDURE Crear_Usuario(IN p_nombre_usuario VARCHAR(30), IN p_email VARCHAR(40), IN p_contraseña_usuario VARCHAR(30), IN p_privilegios_empleado BOOLEAN, IN p_pregunta_seguridad VARCHAR(100), IN p_respuesta_seguridad VARCHAR(100))
BEGIN
    INSERT INTO Usuario(nombre_usuario, email, contraseña_usuario, privilegios_empleado, pregunta_seguridad, respuesta_seguridad) VALUES(p_nombre_usuario, p_email, p_contraseña_usuario, p_privilegios_empleado, p_pregunta_seguridad, p_respuesta_seguridad);
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Usuario(IN p_idusuario INT, IN p_nombre_usuario VARCHAR(30), IN p_email VARCHAR(40), IN p_contraseña_usuario VARCHAR(30), IN p_privilegios_empleado BOOLEAN, IN p_pregunta_seguridad VARCHAR(100), IN p_respuesta_seguridad VARCHAR(100))
BEGIN
    UPDATE Usuario SET nombre_usuario = COALESCE(p_nombre_usuario, nombre_usuario), email = COALESCE(p_email, email), contraseña_usuario = COALESCE(p_contraseña_usuario, contraseña_usuario), privilegios_empleado = COALESCE(p_privilegios_empleado, privilegios_empl
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Usuario()
BEGIN
    SELECT * FROM Usuario;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Usuario(IN p_idusuario INT)
BEGIN
    SELECT * FROM Usuario WHERE IDUsuario = p_idusuario;
END//
DELIMITER ;
```

```
DELIMITER //
CREATE PROCEDURE Buscar_Usuario(IN p_idusuario INT)
BEGIN
    SELECT * FROM Usuario WHERE IDUsuario = p_idusuario;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Usuario(IN p_idusuario INT)
BEGIN
    DELETE FROM Usuario WHERE IDUsuario = p_idusuario;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Usuario_Por_Nombre(IN p_nombre_usuario VARCHAR(30))
BEGIN
    SELECT * FROM Usuario WHERE nombre_usuario = p_nombre_usuario;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Usuarios_Sin_Permisos()
BEGIN
    SELECT * FROM Usuario WHERE privilegios_empleado = FALSE;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Usuarios_Con_Permisos()
BEGIN
    SELECT * FROM Usuario WHERE privilegios_empleado = TRUE;
END//
DELIMITER ;
```

```
DELIMITER //
CREATE PROCEDURE Validar_Usuario_Unico(IN p_nombre_usuario VARCHAR(30))
BEGIN
    SELECT COUNT(*) FROM Usuario WHERE nombre_usuario = p_nombre_usuario;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Validar_Email_Unico(IN p_email VARCHAR(40))
BEGIN
    SELECT COUNT(*) FROM Usuario WHERE email = p_email;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Validar_Usuario_Editar(IN p_nombre_usuario VARCHAR(30), IN p_id INT)
BEGIN
    SELECT COUNT(*) FROM Usuario WHERE nombre_usuario = p_nombre_usuario AND IDUsuario != p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Validar_Email_Editar(IN p_email VARCHAR(40), IN p_id INT)
BEGIN
    SELECT COUNT(*) FROM Usuario WHERE email = p_email AND IDUsuario != p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Contraseña_Usuario(IN p_contraseña_usuario VARCHAR(30), IN p_id INT)
BEGIN
    UPDATE Usuario SET contraseña_usuario = p_contraseña_usuario WHERE IDUsuario = p_id;
END//
DELIMITER ;
```

Buscar_Servicio_Por_Rango_De_Precio: Permite buscar y filtrar los servicios cuyos precios se encuentran dentro de un rango determinado entre un mínimo y un máximo.

Crear_Usuario: Da de alta un nuevo usuario en la base de datos recibiendo como parámetros de entrada su nombre, email, contraseña, privilegios de empleado, pregunta y respuesta de seguridad.

Editar_Usuario: Permite modificar los datos de un usuario existente utilizando la función COALESCE para actualizar únicamente los campos enviados sin afectar a los demás.

Mostrar_Usuario: Devuelve un listado completo con todos los registros de usuarios guardados en la tabla.

Buscar_Usuario: Consulta y trae los datos de un usuario en específico utilizando su identificador único (IDUsuario).

Eliminar_Usuario: Borra un registro de la tabla de usuarios utilizando de forma directa el ID del usuario seleccionado.

Buscar_Usuario_Por_Nombre: Filtra y muestra los datos del usuario que coinciden con un nombre de usuario específico.

Mostrar_Usuarios_Sin_Permisos: Devuelve una lista con todos los usuarios que no poseen privilegios de empleado.

Mostrar_Usuarios_Con_Permisos: Muestra un listado con los usuarios que sí cuentan con privilegios de empleado.

Validar_Usuario_Unico: Cuenta la cantidad de registros que coinciden con un nombre de usuario para verificar su disponibilidad.

Validar_Email_Unico: Cuenta cuántos registros tienen un correo electrónico determinado para comprobar que no se repita.

Validar_Usuario_Editar: Comprueba si un nombre de usuario ya está siendo utilizado por otro usuario diferente durante un proceso de edición.

Validar_Email_Editar: Verifica si un correo electrónico ya se encuentra registrado en otra cuenta distinta al momento de editar los datos.

Editar_Contraseña_Usuario: Actualiza de forma específica la contraseña de un usuario utilizando su identificador único.

22/10/26

Se empezo y termino la tabla pago y se dio inicio a la tabla vuelo

```
-- PROCEDIMIENTOS DE Pago
-- ========================================
DELIMITER //
CREATE PROCEDURE Buscar_Pago_Por_Estado(IN p_estado VARCHAR(20))
BEGIN
    SELECT * FROM Pago WHERE estado = p_estado;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Agregar_Pago(IN p_pasajero_id INT, IN p_boleto_id INT, IN p_costo_servicios DECIMAL(10, 2), IN p_costo_total DECIMAL(10, 2), IN p_estado VARCHAR(20))
BEGIN
    INSERT INTO Pago (pasajero_id, boleto_id, costo_servicios, costo_total, estado) VALUES (p_pasajero_id, p_boleto_id, p_costo_servicios, p_costo_total, p_estado);
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Pagos()
BEGIN
    SELECT * FROM Pago;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pago_Por_Id(IN p_id INT)
BEGIN
    SELECT * FROM Pago WHERE IDPago = p_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Pago(IN p_pasajero_id INT, IN p_boleto_id INT, IN p_costo_servicios DECIMAL(10, 2), IN p_costo_total DECIMAL(10, 2), IN p_estado VARCHAR(20), IN p_id INT)
BEGIN
    UPDATE Pago SET pasajero_id = p_pasajero_id, boleto_id = p_boleto_id, costo_servicios = p_costo_servicios, costo_total = p_costo_total, estado = p_estado WHERE IDPago = p_id;
END //
DELIMITER ;
```

```
DELIMITER //
CREATE PROCEDURE Buscar_Pago_Por_Pasajero(IN p_pasajero_id INT)
BEGIN
    SELECT * FROM Pago WHERE pasajero_id = p_pasajero_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pago_Por_Costo_Mayor_A(IN p_costo_total DECIMAL(10, 2))
BEGIN
    SELECT * FROM Pago WHERE costo_total > p_costo_total;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pago_Por_Costo_Menor_A(IN p_costo_total DECIMAL(10, 2))
BEGIN
    SELECT * FROM Pago WHERE costo_total < p_costo_total;
END//
DELIMITER ;

-- ========================================
-- PROCEDIMIENTOS DE Vuelo
-- ========================================
DELIMITER //
CREATE PROCEDURE Buscar_Vuelo_Por_Id(IN p_idvuelo INT)
BEGIN
    SELECT * FROM Vuelo WHERE IDVuelo = p_idvuelo;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Vuelos_Programados()
BEGIN
    SELECT IDVuelo, avion_id, origen_id, destino_id FROM Vuelo WHERE estado = 'Programado';
END//
DELIMITER ;
```

Buscar_Pago_Por_Estado: Filtra y muestra los pagos que coinciden con un estado específico, como pagado o pendiente.

Agregar_Pago: Da de alta un nuevo pago en la base de datos recibiendo como parámetros el ID del pasajero, el ID del boleto, el costo de los servicios, el costo total y el estado.

Mostrar_Pagos: Devuelve un listado completo con todos los registros de pagos guardados en la tabla.

Buscar_Pago_Por_Id: Consulta y trae los datos de un pago en específico utilizando su identificador único (IDPago).

Editar_Pago: Permite modificar los datos de un pago existente actualizando sus valores y su ID de referencia.

Buscar_Pago_Por_Pasajero: Filtra y muestra todos los pagos que corresponden a un pasajero específico.

Buscar_Pago_Por_Costo_Mayor_A: Devuelve los pagos cuyo costo total supera un monto determinado.

Buscar_Pago_Por_Costo_Menor_A: Consulta los pagos cuyo costo total es inferior a un valor específico.

Buscar_Vuelo_Por_Id: Consulta y trae los datos de un vuelo en concreto utilizando su identificador único.

Mostrar_Vuelos_Programados: Devuelve un listado resumido con los vuelos que se encuentran en estado programado.

Fecha 26/10/26

```
DELIMITER //
CREATE PROCEDURE Agregar_Vuelo(IN p_avion_id VARCHAR(30), IN p_origen_id INT, IN p_destino_id INT, IN p_fecha_hora_salida DATETIME, IN p_fecha_hora_llegada DATETIME, IN p_estado VARCHAR(20))
BEGIN
    INSERT INTO Vuelo (avion_id, origen_id, destino_id, fecha_hora_salida, fecha_hora_llegada, estado) VALUES (p_avion_id, p_origen_id, p_destino_id, p_fecha_hora_salida, p_fecha_hora_llegada, p_estado);
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Todos_Los_Vuelos()
BEGIN
    SELECT * FROM Vuelo;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Vuelo(IN p_avion_id VARCHAR(30), IN p_origen_id INT, IN p_destino_id INT, IN p_fecha_hora_salida DATETIME, IN p_fecha_hora_llegada DATETIME, IN p_estado VARCHAR(20), IN p_id INT)
BEGIN
    UPDATE Vuelo SET avion_id = p_avion_id, origen_id = p_origen_id, destino_id = p_destino_id, fecha_hora_salida = p_fecha_hora_salida, fecha_hora_llegada = p_fecha_hora_llegada, estado = p_estado WHERE IDVuelo = p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Vuelo(IN p_id INT)
BEGIN
    DELETE FROM Vuelo WHERE IDVuelo = p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Vuelo_Por_Avion(IN p_avion_id VARCHAR(30))
BEGIN
    SELECT * FROM Vuelo WHERE avion_id = p_avion_id;
END//
DELIMITER ;
```

```
DELIMITER //
CREATE PROCEDURE Buscar_Vuelo_Por_Destino(IN p_destino_id INT)
BEGIN
    SELECT * FROM Vuelo WHERE destino_id = p_destino_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Vuelo_Por_Estado(IN p_estado VARCHAR(20))
BEGIN
    SELECT * FROM Vuelo WHERE estado = p_estado;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Vuelo_Por_Origen(IN p_origen_id INT)
BEGIN
    SELECT * FROM Vuelo WHERE origen_id = p_origen_id;
END//
DELIMITER ;

-- ========================================
-- PROCEDIMIENTOS DE Pasajero
-- ========================================
DELIMITER //
CREATE PROCEDURE Mostrar_Extras_Pasajeros()
BEGIN
    SELECT IDPasajero, nombre_pasajero, apellido_pasajero FROM Pasajero;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Agregar_Pasajero(IN p_nombre VARCHAR(50), IN p_apellido VARCHAR(50), IN p_dni VARCHAR(15), IN p_telefono VARCHAR(20), IN p_fecha_nacimiento DATE, IN p_nro_pasaporte VARCHAR(20), IN p_pais_emision CHAR(3), IN p_vencimiento_pasaporte DATE, IN p_usua
BEGIN
    INSERT INTO Pasajero (nombre_pasajero, apellido_pasajero, dni_pasajero, telefono, fecha_nacimiento, nro_pasaporte, pais_emision, vencimiento_pasaporte, usuario_id) VALUES (p_nombre, p_apellido, p_dni, p_telefono, p_fecha_nacimiento, p_nro_pasaporte, p_pais_em
END//
DELIMITER ;
```

```
DELIMITER //
CREATE PROCEDURE Mostrar_Pasajeros()
BEGIN
    SELECT * FROM Pasajero;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pasajero_Por_Id(IN p_id INT)
BEGIN
    SELECT * FROM Pasajero WHERE IDPasajero = p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Editar_Pasajero(IN p_nombre VARCHAR(50), IN p_apellido VARCHAR(50), IN p_dni VARCHAR(15), IN p_telefono VARCHAR(20), IN p_fecha_nacimiento DATE, IN p_nro_pasaporte VARCHAR(20), IN p_pais_emision CHAR(3), IN p_vencimiento_pasaporte DATE, IN p_usua
BEGIN
    UPDATE Pasajero SET nombre_pasajero = p_nombre, apellido_pasajero = p_apellido, dni_pasajero = p_dni, telefono = p_telefono, fecha_nacimiento = p_fecha_nacimiento, nro_pasaporte = p_nro_pasaporte, pais_emision = p_pais_emision, vencimiento_pasaporte = p_vencim
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Pasajero(IN p_id INT)
BEGIN
    DELETE FROM Pasajero WHERE IDPasajero = p_id;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pasajero_Por_Nombre(IN p_nombre VARCHAR(50))
BEGIN
    SELECT * FROM Pasajero WHERE nombre_pasajero = p_nombre;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Pasajero_Por_Apellido(IN p_apellido VARCHAR(50))
BEGIN
    SELECT * FROM Pasajero WHERE apellido_pasajero = p_apellido;
END//
DELIMITER ;
```

```sql
DELIMITER //
CREATE PROCEDURE Buscar_Pasajero_Por_Telefono(IN p_telefono VARCHAR(20))
BEGIN
    SELECT * FROM Pasajero WHERE telefono = p_telefono;
END//
DELIMITER ;

-- ========================================
-- PROCEDIMIENTOS DE Boleto
-- ========================================
DELIMITER //
CREATE PROCEDURE Mostrar_Boletos_Para_Checkin()
BEGIN
    SELECT b.IDBoleto FROM Boleto b INNER JOIN Pago pa ON b.IDBoleto = pa.boleto_id LEFT JOIN CheckIn c ON b.IDBoleto = c.boleto_id WHERE pa.estado = 'Pagado' AND c.IDCheckin IS NULL;
END//
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Agregar_Boleto(IN p_pasajero_id INT, IN p_vuelo_id INT, IN p_asiento_id VARCHAR(50), IN p_fecha DATETIME, IN p_precio_base DECIMAL(10, 2), OUT p_id_generado INT)
BEGIN
    INSERT INTO Boleto (pasajero_id, vuelo_id, asiento_id, fecha, precio_base) VALUES (p_pasajero_id, p_vuelo_id, p_asiento_id, p_fecha, p_precio_base);
    SET p_id_generado = LAST_INSERT_ID();
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Mostrar_Boletos()
BEGIN
    SELECT * FROM Boleto;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Boleto_Por_Id(IN p_id INT)
BEGIN
    SELECT * FROM Boleto WHERE IDBoleto = p_id;
END //
DELIMITER ;
```

Agregar_Vuelo: Da de alta un nuevo vuelo en la base de datos recibiendo como parámetros el ID del avión, los IDs de origen y destino, las fechas y horas de salida y llegada, y el estado.

Mostrar_Todos_Los_Vuelos: Devuelve un listado completo con todos los registros de vuelos guardados en la tabla.

Editar_Vuelo: Permite modificar los datos de un vuelo existente actualizando todos sus campos de itinerario y estado mediante su identificador único.

Eliminar_Vuelo: Borra un registro de la tabla de vuelos utilizando de forma directa el ID del vuelo seleccionado.

Buscar_Vuelo_Por_Avion: Filtra y muestra los vuelos que se encuentran asignados a un avión específico.

Buscar_Vuelo_Por_Destino: Filtra y consulta los vuelos cuyo aeropuerto de destino coincide con el ID indicado.

Buscar_Vuelo_Por_Estado: Muestra los vuelos que coinciden con un estado operativo específico, como programado o en vuelo.

Buscar_Vuelo_Por_Origen: Filtra y trae los vuelos que salen desde un aeropuerto de origen determinado.

Mostrar_Extras_Pasajeros: Devuelve un listado resumido con el ID, nombre y apellido de los pasajeros registrados.

Agregar_Pasajero: Da de alta un nuevo pasajero en la base de datos recibiendo sus datos personales, información de pasaporte y su ID de usuario asociado.

Mostrar_Pasajeros: Muestra la lista completa con todos los registros de pasajeros guardados en la tabla.

Buscar_Pasajero_Por_Id: Consulta y trae los datos de un pasajero en específico utilizando su identificador único.

Editar_Pasajero: Permite modificar la información personal y los datos de pasaporte de un pasajero existente utilizando su ID.

Eliminar_Pasajero: Borra un registro de la tabla de pasajeros utilizando de forma directa el identificador del pasajero.

Buscar_Pasajero_Por_Nombre: Filtra y muestra los pasajeros que coinciden con un nombre específico.

Buscar_Pasajero_Por_Apellido: Consulta los pasajeros que comparten un apellido determinado.

Buscar_Pasajero_Por_Telefono: Filtra y muestra el pasajero que coincide con un número de teléfono específico.

Mostrar_Boletos_Para_Checkin: Devuelve una lista con los boletos que ya tienen su pago completado pero todavía no realizaron el check-in.

Agregar_Boletos: Da de alta un nuevo boleto de viaje y devuelve mediante un parámetro de salida el ID generado automáticamente.

Mostrar_Boletos: Muestra el listado completo con todos los registros de boletos guardados en la tabla.

Buscar_Boleto_Por_Id: Consulta y trae los detalles de un boleto en específico utilizando su identificador único.

Fecha 27/10/26

En este dia y en el anterior se hizo hasta la tabla boleto servicio la cual no tiene 10 procedimientos porque no son necesarios al ser una tabla mucho a mucho

```
DELIMITER //
CREATE PROCEDURE Editar_Boleto(IN p_id INT, IN p_pasajero_id INT, IN p_vuelo_id INT, IN p_asiento_id VARCHAR(50), IN p_fecha DATETIME, IN p_precio_base DECIMAL(10, 2))
BEGIN
    UPDATE Boleto SET pasajero_id = p_pasajero_id, vuelo_id = p_vuelo_id, asiento_id = p_asiento_id, fecha = p_fecha, precio_base = p_precio_base WHERE IDBoleto = p_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Boleto(IN p_id INT)
BEGIN
    DELETE FROM Boleto WHERE IDBoleto = p_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Boleto_Por_Pasajero(IN p_pasajero_id INT)
BEGIN
    SELECT * FROM Boleto WHERE pasajero_id = p_pasajero_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Boleto_Por_Vuelo(IN p_vuelo_id INT)
BEGIN
    SELECT * FROM Boleto WHERE vuelo_id = p_vuelo_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Buscar_Boleto_Por_Fecha(IN p_fecha DATETIME)
BEGIN
    SELECT * FROM Boleto WHERE fecha = p_fecha;
END //
DELIMITER ;
```

```
-- ========================================
-- PROCEDIMIENTOS DE Boleto_Servicio
-- ========================================
DELIMITER //
CREATE PROCEDURE Buscar_Servicios_Por_Boleto(IN p_boleto_id INT)
BEGIN
    SELECT s.IDServicio, s.nombre_servicio, s.precio, s.descripcion FROM Servicio s INNER JOIN Boleto_Servicio bs ON s.IDServicio = bs.servicio_id WHERE bs.boleto_id = p_boleto_id;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Insertar_Boleto_Servicio(IN p_boleto_id INT, IN p_servicio_id INT)
BEGIN
    INSERT INTO Boleto_Servicio (boleto_id, servicio_id) VALUES (p_boleto_id, p_servicio_id);
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Eliminar_Servicios_De_Boleto(IN p_boleto_id INT)
BEGIN
    DELETE FROM Boleto_Servicio WHERE boleto_id = p_boleto_id;
END //
DELIMITER ;
```

Editar_Boleto: Permite modificar los datos de un boleto existente actualizando su pasajero, vuelo, asiento, fecha y precio base mediante su identificador único.

Eliminar_Boleto: Borra un registro de la tabla de boletos utilizando de forma directa el ID del boleto seleccionado.

Buscar_Boleto_Por_Pasajero: Filtra y muestra todos los boletos que pertenecen a un pasajero específico.

Buscar_Boleto_Por_Vuelo: Consulta los boletos que se encuentran asociados a un vuelo determinado.

Buscar_Boleto_Por_Fecha: Permite buscar y filtrar los boletos según la fecha y hora exacta de emisión.

Buscar_Servicios_Por_Boleto: Realiza una consulta uniendo las tablas para mostrar todos los servicios adicionales asociados a un boleto específico.

Insertar_Boleto_Servicio: Agrega un nuevo servicio adicional a un boleto mediante la relación de sus respectivos identificadores.

Eliminar_Servicios_De_Boleto: Borra todos los servicios adicionales que estén vinculados a un boleto determinado.

29/10/26

Terminado la ultima tabla se finaiza con procedimientos

```
DELIMITER //
CREATE PROCEDURE Obtener_Datos_Pasajero_Por_Usuario (
    IN p_idusuario INT
)
BEGIN
    SELECT P.IDPasajero, P.nombre_pasajero,
           P.apellido_pasajero, P.dni_pasajero, U.email
    FROM Pasajero P
    INNER JOIN Usuario U ON P.usuario_id = U.IDUsuario
    WHERE U.IDUsuario = p_idusuario;
END //
DELIMITER ;

DELIMITER //
CREATE PROCEDURE Obtener_Asientos_Por_Vuelo (
    IN p_idvuelo INT
)
BEGIN
    SELECT A.codigo, A.disponible
    FROM Asiento A
    INNER JOIN Vuelo V ON A.avion_id = V.avion_id
    WHERE V.IDVuelo = p_idvuelo
    ORDER BY A.codigo;
END //
DELIMITER ;
```

Obtener_Datos_Pasajero_Por_Usuario: Realiza una consulta uniendo las tablas de pasajero y usuario para obtener la información personal y el correo electrónico vinculados a un ID de usuario específico.

Obtener_Asientos_Por_Vuelo: Consulta el código y el estado de disponibilidad de los asientos de un avión relacionándolo con un vuelo determinado, ordenando los resultados por código.

Fecha 05/11/2026

Ese día nos enfrentamos a un contratiempo bastante pesado con la parte administrativa del proyecto. Resulta que fuimos a entrar a Jira, que era la plataforma que veníamos usando para organizar las tareas, ver quién se encargaba de cada cosa y llevar los tiempos, pero nos dimos cuenta de que como habíamos estado tanto tiempo sin iniciar sesión, perdimos el acceso y los

datos que teníamos cargados allí. Como nos quedamos sin esa información clave sobre quién había trabajado en cada cosa y en qué fechas, tuvimos que frenar un momento y ponernos a solucionar el problema de inmediato. Para no perder el registro de todo el esfuerzo que veníamos haciendo, armamos una planilla de Excel desde cero y nos pusimos a reescribir y reestructurar a mano todos los datos, reconstruyendo el historial de las actividades de cada integrante del equipo para volver a dejar por escrito quién hizo qué y cuándo.